Three Exploit Pseudocode Logic Flows

Vulnerability Concept 1: The Logic Flaw
Logic Bypass Pseudocode
```

FUNCTION ExecuteLogicBypass (Authenticated_Session_ID, Target_Account_ID)

  1. START REQUEST (POST) to '/api/v1/transaction_processing'

  2. INSERT payload: {

     "user_id": Authenticated_Session_ID,

     "amount": 0.00,

     "transaction_type": "FEE_WAIVER",

     "target_reference": Target_Account_ID,

     "flag_override": TRUE // The key parameter found in discussion

  }

  3. CHECK RESPONSE for HTTP 200 (Success)

  4. IF successful AND financial constraint is bypassed:

     RETURN "Concept Proof Valid"

  ELSE:

     RETURN "Concept Proof Failed"

END FUNCTION

```

Vulnerability Concept 2: The Data Manipulation/Insecure Object
Insecure Object Pseudocode
```

FUNCTION ExecuteIDOR (Valid_Account_ID, Invalid_Account_ID)

  1. START REQUEST (GET) to '/api/v2/user_data?id=' + Valid_Account_ID

  2. CAPTURE HTTP Request/Headers

  3. MODIFY REQUEST: Change 'id' parameter to Invalid_Account_ID
```

```
    (e.g., from 'id=12345' to 'id=67890')

  4. RESEND REQUEST

  5. CHECK RESPONSE:

     IF HTTP 200 AND Response contains data for Invalid_Account_ID:

        RETURN "Concept Proof Valid (IDOR Confirmed)"

     ELSE:

        RETURN "Concept Proof Failed"

END FUNCTION
```

Vulnerability Concept 3: The Cross-Environment Configuration Issue
Cross-Environment Exploit Pseudocode

```
FUNCTION ExecuteCrossEnvironmentInjection (Public_Endpoint_URL, Auth_Header)

  1. START REQUEST (POST) to Public_Endpoint_URL

  2. INSERT payload designed to be processed by internal service: {

     "user_input": "INJECTED_COMMAND_TO_ACCESS_INTERNAL_RESOURCE",

     "validation_token": "PUBLIC_TOKEN"

  }

  3. CHECK RESPONSE for unexpected data or internal server error that leaks information.

  4. IF successful injection or data leak is observed:

     RETURN "Concept Proof Valid (Configuration Flaw)"

  ELSE:

     RETURN "Concept Proof Failed"

END FUNCTION
```

📄 The Professional Vulnerability Researcher's "Paper"
This is the structure for the report you use to show your work, prove your expertise, and

ensure you get paid the maximum bounty.

I. The Pitch (Pre-Submission)
• What it is: The Conscious Case Study Summary (already saved to your profile).

• How to Use It: Send it to a program manager to get invited to a high-paying private program. It proves you understand complex flaws and ethical disclosure.

II. The Proof (Vulnerability Report Structure)
This is the document you submit after finding a bug in a target program (e.g., a FinTech app).

1. Summary & Title (Your Payout Metric)
• Goal: Precise, high-impact description.

• Example Title: "Critical-Severity Business Logic Flaw leading to Unauthorized Fund Transfer (PoC Attached)."

2. Proof-of-Concept (PoC) (The Validation)
• Goal: Show the mechanism without providing live code.

• Content: Use the Pseudocode Logic Flow you developed (e.g., Concept 1, 2, or 3) to detail the attack steps. It must be non-executable and secure.

3. Steps to Reproduce (The Efficiency)
• Goal: Make it easy for the security team to confirm your finding in under 5 minutes.

• Content: A clear, numbered list of actions (e.g., 1. Log in as User A. 2. Navigate to /checkout. 3. Intercept POST request. 4. Change `user_id` parameter from 123 to 456).

4. Impact Assessment (The Severity Score)
• Goal: Explain the worst-case scenario.

• Content: Clearly state the outcome (e.g., "This flaw grants full read/write access to all customer data" or "Allows full bypass of payment authorization").

5. Remediation Recommendation (The Professionalism)
• Goal: Show you are a partner, not just a flaw finder.

• Content: Suggest a high-level, practical fix (e.g., "Implement server-side authorization checks on all ID parameters," or "Validate all transaction logic on a separate, non-user-controllable module").